Document number: P1032R1
Project: Programming Language C++
Audience: Library Working Group

Antony Polukhin, Yandex.Taxi Ltd, <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2018-10-01

Misc constexpr bits

I. Introduction and Motivation

Members of WG21 and people all around the world were noticing missing `constexpr` in different places of the Standard Library.

This paper attempts to address all the trivial cases in one go.

Adding the `constexpr` is required for progress of constexpr containers, reflection and metaclasses. It also simplifies metaprogramming.

II. Impact on the Standard

This proposal is a pure library extension. It proposes changes to existing headers such that the changes do not break existing code and do not degrade performance. It does not require any changes in the core and it could be implemented in standard C++.

III. Proposed wording relative to N4762

All the additions to the Standard are marked with underlined green.

Green lines are notes for the **editor** that must not be treated as part of the wording.

1. Modifications to "Class template pair" [pairs.pair]

```
namespace std {
    template<class T1, class T2>
    struct pair {
        using first_type = T1;
        using second_type = T2;
        T1 first;
        T2 second;

        pair(const pair&) = default;
        pair(pair&&) = default;

        explicit(see below) constexpr pair();
        explicit(see below) constexpr pair(const T1& x, const T2& y);

        template<class U1, class U2> explicit(see below) constexpr pair(U1&& x, U2&& y);
        template<class U1, class U2> explicit(see below) constexpr pair(const pair<U1, U2>& p);
        template<class U1, class U2> explicit(see below) constexpr pair(pair<U1, U2>&& p);

        template<class... Args1, class... Args2>
        constexpr pair(piecewise_construct_t, tuple<Args1...> first_args, tuple<Args2...> second_args);

        constexpr pair& operator=(const pair& p);

        template<class U1, class U2> constexpr pair& operator=(const pair<U1, U2>& p);
        constexpr pair& operator=(pair&& p) noexcept(see below );

        template<class U1, class U2> constexpr pair& operator=(pair<U1, U2>&& p);
        constexpr void swap(pair& p) noexcept(see below );
    };

    template<class T1, class T2>
    pair(T1, T2) -> pair<T1, T2>;
}
```

All the functions marked with `constexpr` in previous paragraph of this document must be accordingly marked with `constexpr` in detailed description of `pair` functions.

2. Modifications to "Specialized algorithms" [pairs.spec]

```
template<class T1, class T2> constexpr void swap(pair<T1, T2>& x, pair<T1, T2>& y) noexcept(noexcept(x.swap(y)));
    Effects: As if by x.swap(y).
    Remarks: This function shall not participate in overload resolution unless is_swappable_v<T1> is
    true and is_swappable_v<T2> is true.
```

3. Modifications to "Header <tuple> synopsis" [tuple.syn]

```
// 19.5.3.10, specialized algorithms
template<class... Types>
constexpr void swap(tuple<Types...>& x, tuple<Types...>& y) noexcept(see below );
```

4. Modifications to "Class template tuple" [tuple.tuple]

```
namespace std {
    template<class... Types>
    class tuple {
    public:

        // 19.5.3.1, tuple construction
        explicit(see below) constexpr tuple();
        explicit(see below) constexpr tuple(const Types&...);

        template<class... UTypes>
        explicit(see below) constexpr tuple(UTypes&&...);

        tuple(const tuple&) = default;
        tuple(tuple&&) = default;

        template<class... UTypes>
        explicit(see below) constexpr tuple(const tuple<UTypes...>&);

        template<class... UTypes>
        explicit(see below) constexpr tuple(tuple<UTypes...>&&);

        template<class U1, class U2>
        explicit(see below) constexpr tuple(const pair<U1, U2>&);

        template<class U1, class U2>
        explicit(see below) constexpr tuple(pair<U1, U2>&&);

        // allocator-extended constructors
        template<class Alloc>
        constexpr tuple(allocator_arg_t, const Alloc& a);
        template<class Alloc>
        explicit(see below) constexpr tuple(allocator_arg_t, const Alloc& a, const Types&...);
        template<class Alloc, class... UTypes>
        explicit(see below) constexpr tuple(allocator_arg_t, const Alloc& a, UTypes&&...);

        template<class Alloc>
        constexpr tuple(allocator_arg_t, const Alloc& a, const tuple&);
        template<class Alloc>
        constexpr tuple(allocator_arg_t, const Alloc& a, tuple&&);
        template<class Alloc, class... UTypes>
        explicit(see below) constexpr tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
        template<class Alloc, class... UTypes>
        explicit(see below) constexpr tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
        template<class Alloc, class U1, class U2>
        explicit(see below) constexpr tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
        template<class Alloc, class U1, class U2>
        explicit(see below) constexpr tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);

        // 19.5.3.2, tuple assignment
        constexpr tuple& operator=(const tuple&);
        constexpr tuple& operator=(tuple&&) noexcept(see below );
        template<class... UTypes>
        constexpr tuple& operator=(const tuple<UTypes...>&);
        template<class... UTypes>
        constexpr tuple& operator=(tuple<UTypes...>&&);
        template<class U1, class U2>
        constexpr tuple& operator=(const pair<U1, U2>&);
        template<class U1, class U2>
        constexpr tuple& operator=(pair<U1, U2>&&);

        // 19.5.3.3, tuple swap
        constexpr void swap(tuple&) noexcept(see below );
    };
```

All the functions marked with `constexpr` in previous paragraph of this document must be accordingly marked with `constexpr` in [tuple.cnstr], [tuple.assign], [tuple.swap].

5. Modifications to "Header <array> synopsis" [array.syn]

```
#include <initializer_list>

namespace std {
    // 21.3.7, class template array
    template<class T, size_t N> struct array;

    ...

    template<class T, size_t N>
      constexpr void swap(array<T, N>& x, array<T,N>& y) noexcept(noexcept(x.swap(y)));
```

```
    ...
}
```

6. Modifications to "Class template array overview" [array.overview]

```
    template<class T, size_t N>
    struct array {

        ...

        constexpr void fill(const T& u);
        constexpr void swap(array&) noexcept(is_nothrow_swappable_v<T>);

        ...
    };
```

7. Modifications to "array member functions" [array.members]

```
constexpr void fill(const T& u);
    Effects: As if by fill_n(begin(), N, u).

constexpr void swap(array& y) noexcept(is_nothrow_swappable_v<T>);
    Effects: Equivalent to swap_ranges(begin(), end(), y.begin()).
    [ Note: Unlike the swap function for other containers, array::swap takes linear time, may exit via an
    exception, and does not cause iterators to become associated with the other container. — end note ]
```

8. Modifications to "array specialized algorithms" [array.special]

```
template<class T, size_t N>
constexpr void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));
    Remarks: This function shall not participate in overload resolution unless N == 0 or is_swappable_v<T>
    is true.
    Effects: As if by x.swap(y).
    Complexity: Linear in N.
```

9. Modifications to "General container requirements" ("Optional container operations") [container.requirements.general]

```
    Table 66 lists operations that are provided for some types of containers but not others. Those containers for
    which the listed operations are provided shall implement the semantics described in Table 66 unless otherwise
    stated. If iterators passed to lexicographical_compare()
    satisfy the constexpr iterator requirement then the operations described in Table 66 are constexpr.
```

10. Modifications to all the char_traits specializations [char.traits.specializations.*]

```
    static constexpr char_type* move(char_type* s1, const char_type* s2, size_t n);
    static constexpr char_type* copy(char_type* s1, const char_type* s2, size_t n);
    static constexpr char_type* assign(char_type* s, size_t n, char_type a);
```

All the functions marked with `constexpr` in previous paragraph of this document must be accordingly marked with `constexpr` in [char.traits.specializations.*].

11. Modifications to "Class template basic_string_view" [string.view.template]

```
    // 20.4.2.6, string operations
    constexpr size_type copy(charT* s, size_type n, size_type pos = 0) const;
```

12. Modifications to "String operations" [string.view.ops]

```
constexpr size_type copy(charT* s, size_type n, size_type pos = 0) const;
    Let rlen be the smaller of n and size() - pos.
    Throws: out_of_range if pos > size().
    Requires: [s, s + rlen) is a valid range.
    Effects: Equivalent to traits::copy(s, data() + pos, rlen).
    Returns: rlen.
    Complexity: O(rlen).
```

13. Modifications to "Class template default_searcher" [func.search.default]

```
template<class ForwardIterator1, class BinaryPredicate = equal_to<>>
class default_searcher {
public:
    constexpr default_searcher(ForwardIterator1 pat_first, ForwardIterator1 pat_last, BinaryPredicate pred = BinaryPredicate());

    template<class ForwardIterator2>
    constexpr pair<ForwardIterator2, ForwardIterator2>
    operator()(ForwardIterator2 first, ForwardIterator2 last) const;
private:
    ForwardIterator1 pat_first_;
    ForwardIterator1 pat_last_;
    BinaryPredicate pred_;
};
```

```
constexpr default_searcher(ForwardIterator pat_first, ForwardIterator pat_last, BinaryPredicate pred = BinaryPredicate());
    Effects: Constructs a default_searcher object, initializing pat_first_ with pat_first, pat_last_
    with pat_last, and pred_ with pred.
    Throws: Any exception thrown by the copy constructor of BinaryPredicate or ForwardIterator1.

template<class ForwardIterator2>
constexpr pair<ForwardIterator2, ForwardIterator2>
operator()(ForwardIterator2 first, ForwardIterator2 last) const;
    Effects: Returns a pair of iterators i and j such that
    (3.1) — i == search(first, last, pat_first_, pat_last_, pred_), and
    (3.2) — if i == last, then j == last, otherwise j == next(i, distance(pat_first_, pat_last_)).
```

14. Modifications to "Header <iterator> synopsis" [iterator.synopsis]

```
    template<class Container> class back_insert_iterator;
    template<class Container>
    constexpr back_insert_iterator<Container> back_inserter(Container& x);

    template<class Container> class front_insert_iterator;
    template<class Container>
    constexpr front_insert_iterator<Container> front_inserter(Container& x);

    template<class Container> class insert_iterator;
    template<class Container>
    constexpr insert_iterator<Container> inserter(Container& x, typename Container::iterator i);
```

15. Modifications to "Class template back_insert_iterator" [back.insert.iterator]

```
namespace std {
    template<class Container>
    class back_insert_iterator {
    protected:
        Container* container;
    public:
        using iterator_category = output_iterator_tag;
        using value_type = void;
        using difference_type = void;
        using pointer = void;
        using reference = void;
        using container_type = Container;


        explicit constexpr back_insert_iterator(Container& x);
        constexpr back_insert_iterator& operator=(const typename Container::value_type& value);
        constexpr back_insert_iterator& operator=(typename Container::value_type&& value);

        constexpr back_insert_iterator& operator*();
        constexpr back_insert_iterator& operator++();
        constexpr back_insert_iterator operator++(int);
    };

    template<class Container>
    constexpr back_insert_iterator<Container> back_inserter(Container& x);
}
```

All the functions marked with `constexpr` in previous paragraph of this document must be accordingly marked with `constexpr` in [back.insert.iter.*] and [back.inserter].

16. Modifications to "Class template front_insert_iterator" [front.insert.iterator]

```
namespace std {
    template<class Container>
    class front_insert_iterator {
    protected:
        Container* container;
    public:
        using iterator_category = output_iterator_tag;
        using value_type = void;
        using difference_type = void;
        using pointer = void;
        using reference = void;
        using container_type = Container;

        explicit constexpr front_insert_iterator(Container& x);
        constexpr front_insert_iterator& operator=(const typename Container::value_type& value);
        constexpr front_insert_iterator& operator=(typename Container::value_type&& value);

        constexpr front_insert_iterator& operator*();
        constexpr front_insert_iterator& operator++();
        constexpr front_insert_iterator operator++(int);
    };

    template<class Container>
    constexpr front_insert_iterator<Container> front_inserter(Container& x);
}
```

All the functions marked with `constexpr` in previous paragraph of this document must be accordingly marked with `constexpr` in [front.insert.iter.*] and [front.inserter].

17. Modifications to "Class template insert_iterator" [insert.iterator]

```
namespace std {
    template<class Container>
    class insert_iterator {
    protected:
        Container* container;
        typename Container::iterator iter;
    public:
        using iterator_category = output_iterator_tag;
        using value_type = void;
        using difference_type = void;
        using pointer = void;
        using reference = void;
        using container_type = Container;

        explicit constexpr insert_iterator(Container& x, typename Container::iterator i);
        constexpr insert_iterator& operator=(const typename Container::value_type& value);
        constexpr insert_iterator& operator=(typename Container::value_type&& value);

        constexpr insert_iterator& operator*();
        constexpr insert_iterator& operator++();
        constexpr insert_iterator& operator++(int);
    };

    template<class Container>
    constexpr insert_iterator<Container> inserter(Container& x, typename Container::iterator i);
}
```

All the functions marked with `constexpr` in previous paragraph of this document must be accordingly marked with `constexpr` in [insert.iter.*] and [inserter].

18. Feature-testing macro

Add a row into the "Standard library feature-test macros" table [support.limits.general]:

__cpp_lib_constexpr_misc	201811	<array> <functional> <iterator> <string_view> <tuple> <utility>

IV. Revision History

Revision 1:

- Removed additionas that conflict with P1023.
- Rebased on N4762.
- Updated the "Feature-testing macro" section.

Revision 0:

- Initial proposal.

V. Acknowledgements

Many thanks to people who pointed me on some of the missing bits: Alisdair Meredith, Ben Deane, Casey Carter, Jason Turner, Louis Dionne, Marshall Clow.